

Simple Scheduling Simulation

Cha Hohyeon [32224560]
Nathaniel Vallen [32221415]
Global SW Convergence

List of Contents

Introduction.....	2
Important Concepts & Considerations.....	5
Build Configurations & Environment.....	9
Additional Acknowledgments and Notes.....	10
Screen Captures.....	10
Conclusion.....	13

Introduction

This project has a goal of making a program which simulates a simple scheduling scheme. Scheduling can be understood as the theory and practice of deciding what to do next. In the scope of operating systems, OS performs scheduling by dispatching and removing processes from the CPU. In order to understand this better, we need to first know the life cycle of a process.

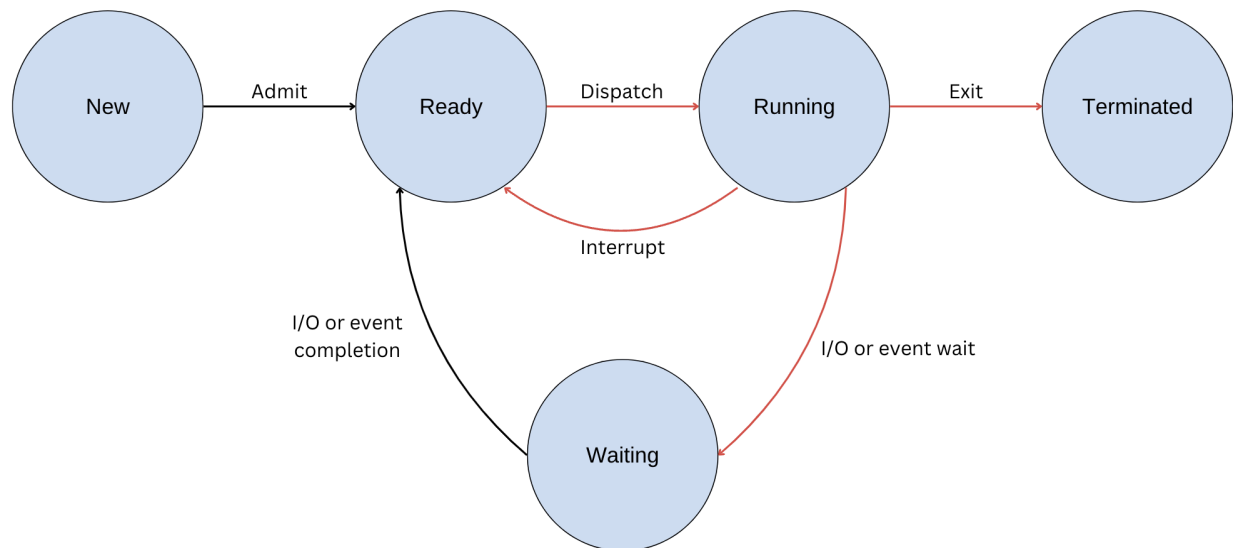


Figure 1. Life cycle of a process

The diagram above, also called the “5-state diagram”, describes the states of a process since it was created until its termination. When a process is first created, it starts off in a “new” state. In this state, the address space creation and CPU state initialization take place. After that, the process gets admitted and enters the “ready” state. The “ready” state is often represented as a queue, known as “ready queue” or “run queue”, which as the name describes, is a queue that holds processes which are ready to run and are waiting to be assigned by the scheduler. Once a process gets dispatched, it will enter the “running” state. At this stage, the CPU context is loaded into the physical CPU, allowing the process to run. The state that comes after this depends on what happened to the running process. If a running process encounters any interrupt, then it returns to “ready” state. Another possibility is that the running process triggers a system call (triggered by an event or I/O job), in which case, it enters the “waiting” state. Here, it waits until the said event or I/O job is done, after which it goes back to the “ready” state. The last possible case for a running process is for it to exit, entering the “terminated” state, hence ending its life cycle.

Since scheduling involves dispatching and removing processes, it occurs whenever the “running” state is involved. This includes when a ready process is dispatched (ready-running),

when a running process is interrupted (running-ready), when a running process waits for an I/O job or a specific event (running-waiting), and when a running process exits (running-terminated). In addition, scheduling follows a specific pattern or algorithm in deciding which process to dispatch next. This algorithm is also known as a scheduling policy. There are several types of scheduling policies, including First In First Out (FIFO/FCFS), Shortest Job First (SJF), and Round Robin (RR).

In FIFO, the order of scheduling follows the order of their arrivals in the queue. In other words, the process that enters the queue first, gets scheduled first. For example, if we have 5 tasks that enter the queue in order from 0 to 4 with the interval of 1 second each, then we would have:

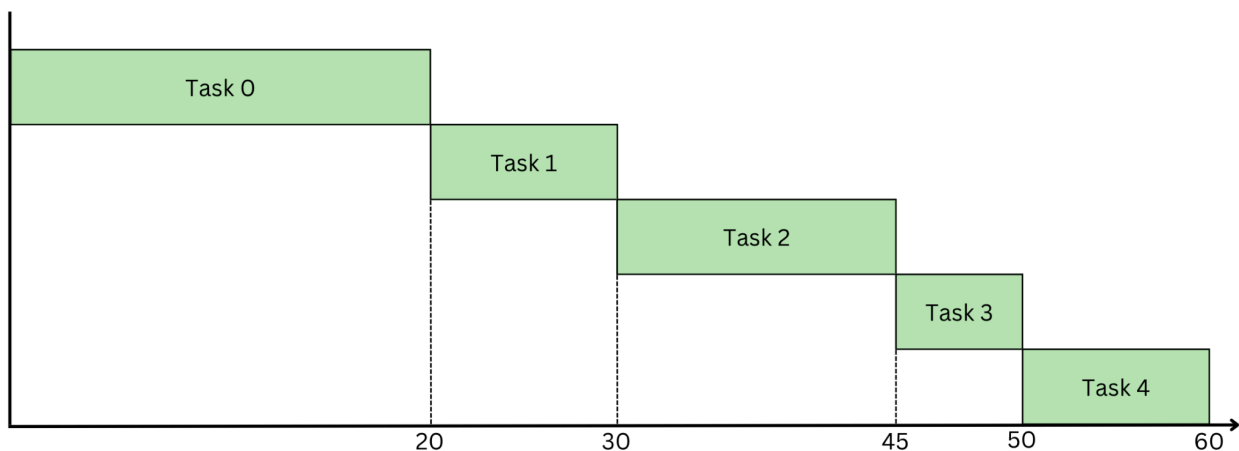


Figure 2. Gantt chart of a FIFO scheduling

While this can be the simplest and easiest implementation of scheduling, FIFO scheduling has some weaknesses regarding average waiting time. Since the first job is not guaranteed to be the shortest job, in the case of the first job taking a lot of time, the waiting time of other processes would rise significantly.

On the other hand, the SJF policy guarantees that the shortest job gets scheduled first and continues in ascending order when all processes enter the queue at the same time. However, this case is not practical as in most cases, there are intervals between process entries in the ready queue. Continuing our assumption from earlier, the result of SJF scheduling policy would look something like this:

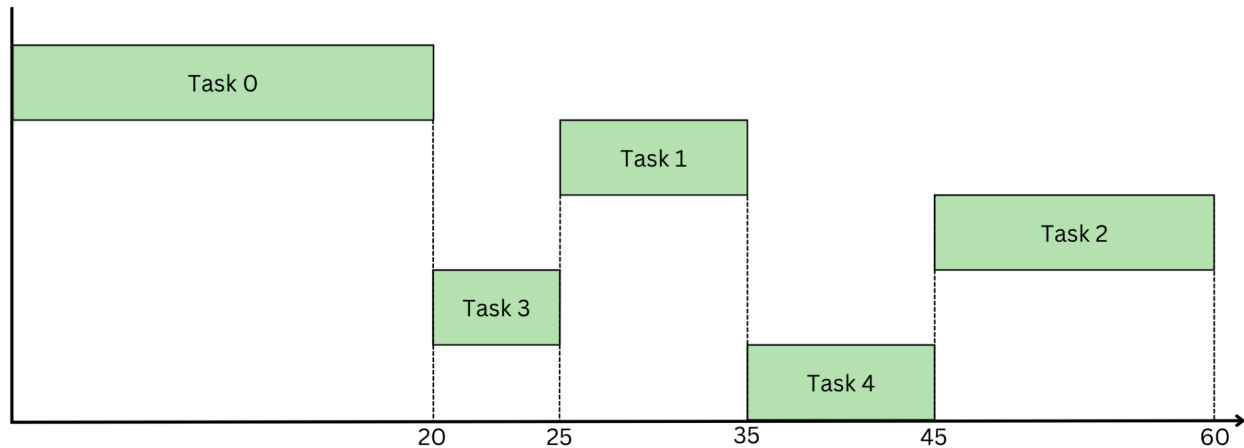


Figure 3. Gantt chart of an SJF scheduling

Take note that the shortest job in this program is task 3, yet the first task to be executed is task 0. Since task 0 enters the ready queue first, then by that time, the OS has no other process to compare task 0 to, resulting in task 0 being considered as the shortest job available at that point of time. However, after the 3rd second, task 1, 2, 3, and 4 have entered the queue. Hence by the time task 0 finishes, we will have all of the other processes to compare and choose the shortest job from.

The last scheduling that we will introduce is the round robin policy. For this policy, we need to first set a value called time quantum. The round robin policy assigns each process a set of time quantum (or time slice) within which a process should finish its job. If the process is unable to finish its job within the given time quantum, then the job is halted and another process is given the turn to do their job. This pattern repeats until every job is done. In our example, if the time quantum is 5 seconds, then a round robin process would give the following result:

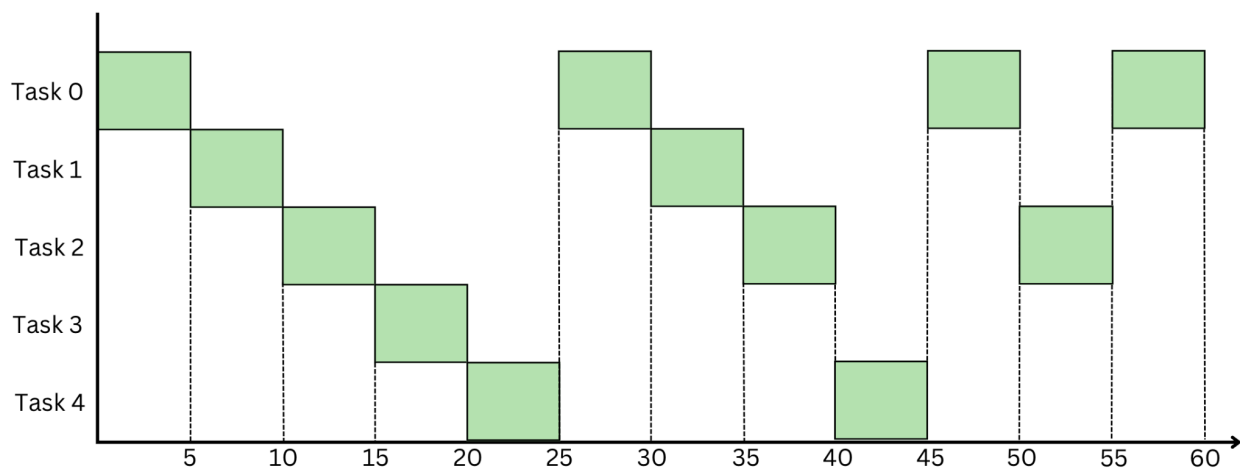


Figure 4. Gantt chart of a RR scheduling with 5-second time quantum

In determining the value of the time quantum, we must be careful as to not make it too large or too small. An overly large time quantum will compromise the system's response time, while an excessively small one will reduce the throughput and efficiency of CPU resource usage by going through unnecessary context switches.

Before going in depth into the details of the project, there are some key concepts related to the operation of the program. Firstly, a timer is set to imitate the behavior of a timer tick by making the parent process receive an ALARM interrupt signal every tick. Secondly, there is a run and a wait queue which hold child processes. The run queue contains child processes which are in "ready" state, while the wait queue contains the child processes in "waiting" state. Third, the interprocess communication (IPC) between the parent and child process occurs via the signal() function. IPC occurs in several cases, such as when a child process is done with CPU or I/O burst and when a parent process needs to command the child process to decrement their CPU burst (simulating some work). Some types of signals involved in this project include SIGALRM, SIGINT, SIGTERM, SIGUSR1, and SIGUSR2.

In the start of this project, the parent process forks 10 child processes, each of which has a randomized number of CPU bursts and I/O bursts (ranging from 1 to 10). Each child process must finish their CPU job before starting on their I/O job. Upon the completion of both CPU and I/O jobs, a child process gets new values of CPU and I/O bursts reassigned to them (also random values). The scheduling policy used in this project is round robin. The details of time quantum and timer tick used will be discussed in a later section. With round robin scheduling, the child process which is created first gets a time slice from the parent process. During this time slice, a child process must do its job and try to finish it. The parent process keeps track of how much time remains for the child process. Once the time is up, then the child process stops its work and gets scheduled out by the parent process and another child process is given the turn to do their job. This way, round robin scheduling policy ensures fair time distribution among all child processes. Once a child process completes its CPU burst, it is moved from the run queue to the waiting queue, where it decrements its I/O burst every timer tick (simulating I/O job). After both CPU and I/O bursts are done, the child process gets dequeued and re-enqueued with random values of CPU and I/O bursts. This sequence repeats until 10.000 time ticks have passed.

Important Concepts & Considerations

- I. The workflow of the program can be drawn as shown below:

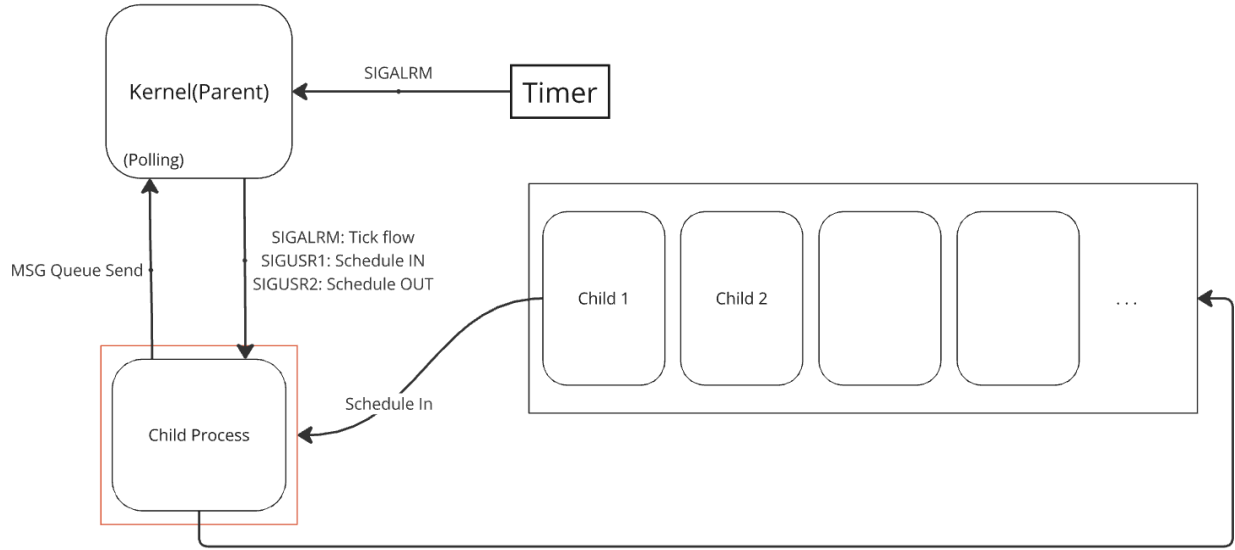


Figure 5. Project workflow

II. The program is comprised of several files:

Name	Description
child.c	Contain signal handlers and I/O request IPC for child processes.
scheduler.c	Contain interrupt, alarm, and I/O schedule handler, alongside scheduler initiation and run code.
message_queue.c	Contain message queue initiation and termination.
pid_queue.c	Contain initiation for PCB, PCB queue, and other queue-related operations.
process.c	Contain details for launching the scheduler, child, and parent process.
timer.c	Contain timer initiation and termination.
main.c	Contain result-determining declarations and launcher trigger.

III. Several important functions are also used in this project, including:

Name	Return Type	Details
fork()	pid_t	Create a new process by duplicating the

		calling process. A child process returns 0, a parent process returns the child process's PID, and an error returns -1.
getpid()	pid_t	Return the PID of the calling process.
kill(pid_t, int)	int	Send a signal to a process and return success status.
msgget(key_t, int)	int	Create/return a message queue identifier.
msgrcv(int, void*, size_t, long, int)	ssize_t	Read a message from a message queue and place it in a user-defined buffer.
setitimer(int, const struct itimerval*, struct itimerval*)	int	Set a timer (depending on the type of timer).
sigaction(int, const struct sigaction*, struct sigaction*)	int	Change the action taken by a process once a specific signal is received.
signal(int, sighandler_t)	void	Send a signal to a signal handler.
waitpid(pid_t, int*, int)	pid_t	Wait for a specific process's status information for a process termination.

IV. Below are the name and definitions of each signal used in this project:

Name	Definition
SIGALRM	A signal for timer events.
SIGINT	A signal (from the user) to terminate a process.
SIGTERM	A signal (from the program) to terminate a process.
SIGUSR1	A signal to indicate that the process is scheduled in.
SIGUSR2	A signal to indicate that the process is scheduled out.

- V. The round robin scheduling policy used has a timer tick of 0.01 second and time quantum of 5 time ticks. The main reason for picking these numbers is to balance the length of the time quantum as to not make it too long or too short. With the time limit of 10.000 ticks, the program runs for exactly 100 seconds (1 minute and 40 seconds).
- VI. For the sake of better understanding, “run queue” is referred to as “ready queue” in the code.
- VII. In the output, the details about the child process are written in the format of *[PID (CPU_burst:IO_burst)]*.
- VIII. Some scheduling metrics are used to compare and see how using different time quanta can have an effect on the efficiency of the program.
 - A. Waiting Time
 - The total time the job spent in the ready queue.
 - B. Response Time
 - The total time between the job’s arrival in the ready queue and the job’s first scheduling dispatch.
 - C. Execution Time
 - The total time spent by the process doing the job (not idle).
 - D. Completion Time
 - The total time between the job’s arrival in the ready queue and job’s completion.

The relationship between the metrics can be understood as follows:

$$\textit{Completion Time} = \textit{Response Time} + \textit{Waiting Time} + \textit{Execution Time}$$

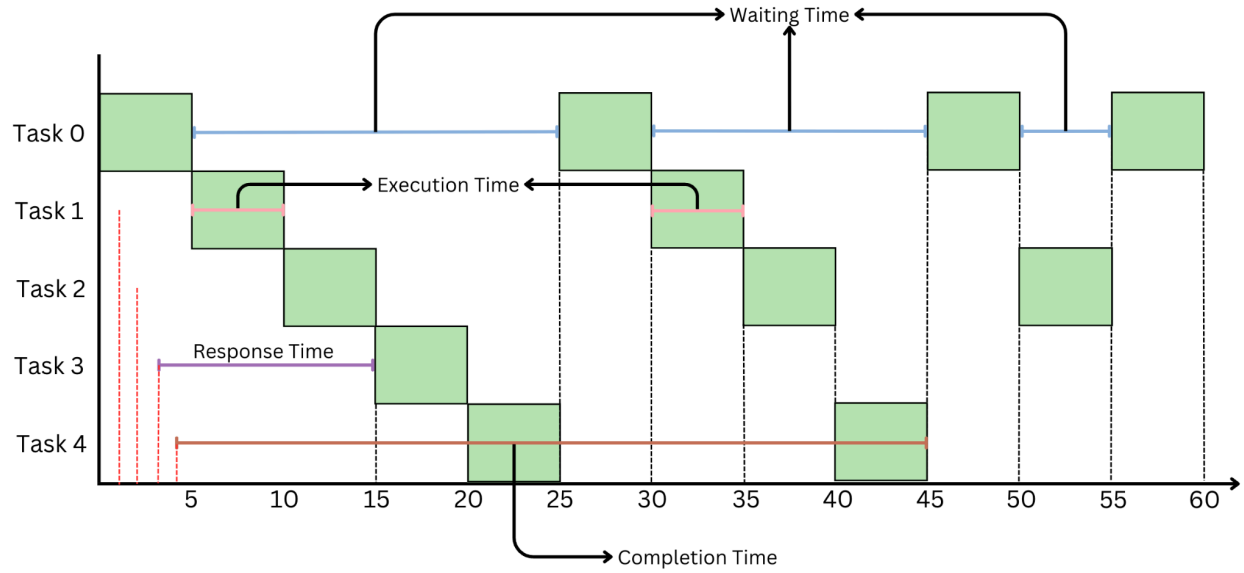


Figure 6. Range of waiting time, response time, execution time, and completion time (the red line marks the job's time of entry)

IX. In this project, we do not take an extra time tick for the scheduler to dispatch, remove, or perform any operations with the processes. In other words, context switch overhead is completely ignored. So to calculate the performance metrics, we can count how many context switch overhead occurred within the 10.000 time ticks range. Logically, more context switching means higher context switching overhead, hence slowing down the completion time in real life cases. The formula related to the overhead can be described as:

$$Number\ of\ Context\ Switch = \frac{Total\ Tick}{Time\ Quantum} - 1$$

This is derived from the fact that a context switch occurs every time quantum (every time a time slice is up) except for the last time quantum (where the program terminates instead of performing a context switch).

Build Configurations & Environment

The code in this project was written in C programming language using JetBrains CLion IDE.

Additional Acknowledgments and Notes

- The assistance of AI-based tools were used in the process of correcting errors in child.c and scheduler.c.
- The codes from pid_queue.c were also generated by AI-based tools.
- Currently, there are some errors in the calculation of completion time (in the metrics section). For the proper calculation formula, refer to page 8.
- Regarding the submission of the code, the relevant code is pushed to the branch “32224560_hohyeon”.

Screen Captures

```
[Tick:: 0]
[Scheduler] No current process
Ready Queue: [10251(0:0)] [10252(0:0)] [10253(0:0)] [10254(0:0)] [10255(0:0)] [10256(0:0)] [10257(0:0)] [10258(0:0)] [10259(0:0)] [10260(0:0)]
Queue is empty.
=====
[Scheduler] Schedule In 10251
=====
[Tick:: 0]
[Scheduler] Current Process = [10251(0:0)]
Ready Queue: [10252(0:0)] [10253(0:0)] [10254(0:0)] [10255(0:0)] [10256(0:0)] [10257(0:0)] [10258(0:0)] [10259(0:0)] [10260(0:0)]
Queue is empty.
[Tick:: 1]
[Scheduler] Current Process = [10251(-1:0)]
Ready Queue: [10252(0:0)] [10253(0:0)] [10254(0:0)] [10255(0:0)] [10256(0:0)] [10257(0:0)] [10258(0:0)] [10259(0:0)] [10260(0:0)]
Queue is empty.
=====
[Scheduler] Schedule In 10252
=====
[Tick:: 1]
[Scheduler] Current Process = [10252(0:0)]
Ready Queue: [10253(0:0)] [10254(0:0)] [10255(0:0)] [10256(0:0)] [10257(0:0)] [10258(0:0)] [10259(0:0)] [10260(0:0)]
Waiting Queue: [10251(X:7)]

[Tick:: 2]
[Scheduler] Current Process = [10252(-1:0)]
Ready Queue: [10253(0:0)] [10254(0:0)] [10255(0:0)] [10256(0:0)] [10257(0:0)] [10258(0:0)] [10259(0:0)] [10260(0:0)]
Waiting Queue: [10251(X:6)]
```

Figure 7. Initial queue at tick 0, displaying the PID and their corresponding CPU and I/O bursts

```
[Tick:: 9998]
[Scheduler] Current Process = [10254(4:13)]
Ready Queue: [10253(14:12)] [10260(7:20)] [10259(2:20)] [10252(9:12)] [10255(19:12)] [10258(2:20)] [10256(2:20)]
Waiting Queue: [10251(X:2)] [10257(X:17)]

[Tick:: 9999]
[Scheduler] Current Process = [10254(3:13)]
Ready Queue: [10253(14:12)] [10260(7:20)] [10259(2:20)] [10252(9:12)] [10255(19:12)] [10258(2:20)] [10256(2:20)]
Waiting Queue: [10251(X:1)] [10257(X:16)]

[Scheduler::10249] Schedule out [Currentprocess = 10254]
=====
[Scheduler] Schedule In 10253
=====
[Tick:: 10000]
[Scheduler] Current Process = [10253(14:12)]
Ready Queue: [10260(7:20)] [10259(2:20)] [10252(9:12)] [10255(19:12)] [10258(2:20)] [10256(2:20)] [10251(19:12)] [10254(2:13)]
Waiting Queue: [10257(X:15)]
```

Figure 8. Termination at tick 10.000

```

=====
[Scheduler] Schedule In 10251
=====
[Tick:: 10]
[Scheduler] Current Process = [10251(10:6)]
Ready Queue: [10252(10:6)] [10253(10:6)] [10254(10:6)] [10255(10:6)] [10256(10:6)]
Waiting Queue: [10257(X:1)] [10258(X:3)] [10259(X:5)] [10260(X:7)]

[Tick:: 11]
[Scheduler] Current Process = [10251(9:6)]
Ready Queue: [10252(10:6)] [10253(10:6)] [10254(10:6)] [10255(10:6)] [10256(10:6)] [10257(10:6)]
Waiting Queue: [10258(X:2)] [10259(X:4)] [10260(X:6)]

[Tick:: 12]
[Scheduler] Current Process = [10251(8:6)]
Ready Queue: [10252(10:6)] [10253(10:6)] [10254(10:6)] [10255(10:6)] [10256(10:6)] [10257(10:6)]
Waiting Queue: [10258(X:1)] [10259(X:3)] [10260(X:5)]

[Tick:: 13]
[Scheduler] Current Process = [10251(7:6)]
Ready Queue: [10252(10:6)] [10253(10:6)] [10254(10:6)] [10255(10:6)] [10256(10:6)] [10257(10:6)] [10258(10:6)]
Waiting Queue: [10259(X:2)] [10260(X:4)]

[Tick:: 14]
[Scheduler] Current Process = [10251(6:6)]
Ready Queue: [10252(10:6)] [10253(10:6)] [10254(10:6)] [10255(10:6)] [10256(10:6)] [10257(10:6)] [10258(10:6)]
Waiting Queue: [10259(X:1)] [10260(X:3)]

[Scheduler::10249] Schedule out [Current process = 10251]

```

Figure 9. A process doing CPU burst (work) and gets scheduled out once the time slice is up

```

=====
[Scheduler] Schedule In 10252
=====
[Tick:: 2831]
[Scheduler] Current Process = [10252(3:1)]
Ready Queue: [10260(2:4)] [10255(8:1)] [10254(2:4)] [10251(3:1)] [10256(8:1)] [10257(8:1)] [10259(2:4)] [10253(3:1)]
Waiting Queue: [10258(X:4)]

[Tick:: 2832]
[Scheduler] Current Process = [10252(2:1)]
Ready Queue: [10260(2:4)] [10255(8:1)] [10254(2:4)] [10251(3:1)] [10256(8:1)] [10257(8:1)] [10259(2:4)] [10253(3:1)]
Waiting Queue: [10258(X:3)]

[Tick:: 2833]
[Scheduler] Current Process = [10252(1:1)]
Ready Queue: [10260(2:4)] [10255(8:1)] [10254(2:4)] [10251(3:1)] [10256(8:1)] [10257(8:1)] [10259(2:4)] [10253(3:1)]
Waiting Queue: [10258(X:2)]

[Tick:: 2834]
[Scheduler] Current Process = [10252(0:1)]
Ready Queue: [10260(2:4)] [10255(8:1)] [10254(2:4)] [10251(3:1)] [10256(8:1)] [10257(8:1)] [10259(2:4)] [10253(3:1)]
Waiting Queue: [10258(X:1)]

```

Figure 10. A process waiting for I/O burst completion

```

=====
[Scheduler] Schedule In 10253
=====
[Tick:: 2786]
[Scheduler] Current Process = [10253(2:4)]
Ready Queue: [10258(7:4)] [10252(8:1)] [10260(7:4)] [10255(2:4)] [10254(7:4)] [10251(8:1)] [10256(2:4)] [10257(2:4)]
Waiting Queue: [10259(X:2)]

[Tick:: 2787]
[Scheduler] Current Process = [10253(1:4)]
Ready Queue: [10258(7:4)] [10252(8:1)] [10260(7:4)] [10255(2:4)] [10254(7:4)] [10251(8:1)] [10256(2:4)] [10257(2:4)]
Waiting Queue: [10259(X:1)]

[Tick:: 2788]
[Scheduler] Current Process = [10253(0:4)]
Ready Queue: [10258(7:4)] [10252(8:1)] [10260(7:4)] [10255(2:4)] [10254(7:4)] [10251(8:1)] [10256(2:4)] [10257(2:4)] [10259(7:4)]
Queue is empty.
=====
[Scheduler] Schedule In 10258
=====
[Tick:: 2788]
[Scheduler] Current Process = [10258(7:4)]
Ready Queue: [10252(8:1)] [10260(7:4)] [10255(2:4)] [10254(7:4)] [10251(8:1)] [10256(2:4)] [10257(2:4)] [10259(7:4)]
Waiting Queue: [10253(X:4)]

```

Figure 11. A process goes into the waiting queue once its CPU burst is finished

```

=====
[Scheduler] Schedule In 10259
=====
[Tick:: 3080]
[Scheduler] Current Process = [10259(9:1)]
Ready Queue: [10253(4:14)] [10258(4:1)] [10260(4:1)] [10255(4:14)] [10252(19:15)] [10254(4:1)] [10256(4:14)]
Waiting Queue: [10251(X:4)] [10257(X:1)]

[Tick:: 3081]
[Scheduler] Current Process = [10259(8:1)]
Ready Queue: [10253(4:14)] [10258(4:1)] [10260(4:1)] [10255(4:14)] [10252(19:15)] [10254(4:1)] [10256(4:14)] [10257(4:14)]
Waiting Queue: [10251(X:3)]

```

Figure 12. A process getting reassigned with new CPU and I/O bursts once its I/O burst is finished

```

Process 10256 Metrics:
  Waiting Time: 86
  Response Time: 31
  Execution Time: 5
  Completion Time: 43
Process 10251 Metrics:
  Waiting Time: 3
  Response Time: 0
  Execution Time: 0
  Completion Time: 1
Process 10254 Metrics:
  Waiting Time: 65
  Response Time: 28
  Execution Time: 5
  Completion Time: 33
Process 10253 Metrics:
  Waiting Time: 62
  Response Time: 24
  Execution Time: 7
  Completion Time: 60

```

Figure 13. Process metrics (in ticks)

Conclusion

The project simulates a simple scheduling done by the OS in machines in real life. For simplicity, the CPU and I/O jobs are replaced with simple variable decrementations. The project demonstrates how round robin scheduling implements different queues and IPC methods to manage fair CPU resource distribution for all processes. The project also showcases the effect of choosing different timer ticks and timer quanta on a machine's efficiency in process scheduling. This can be seen by the aforementioned metrics (completion time, waiting time, etc.) and the number of context switches done within the 10.000 time ticks period.